

Java String Operations Lab Manual

Comprehensive Documentation & Technical Report for 15 Fundamental String Programs

Course Topic	Java Programming Lab – String Handling Class Operations
Target Platform	Java SE Environment (JDK 8 or higher)
Document Scope	Objectives, System Requirements, Algorithm, Class UML Diagrams, Code, Compilation & Execution Steps, and Output Analysis for 15 Java Programs

SYSTEM REQUIREMENTS SPECIFICATION

Component	Minimum Hardware Requirement	Software & Environment Requirement
Processor	Intel Core i3 / AMD Ryzen 3 or equivalent (1.5 GHz)	JDK 8 (Java Development Kit) or higher
Memory (RAM)	2 GB RAM minimum (4 GB recommended)	Windows 10/11, macOS, or Linux OS
Storage	500 MB available disk space for JDK & source files	VS Code, Eclipse, IntelliJ IDEA, or Terminal / Command Prompt
Architecture	x86_64 / ARM64 execution platform	`javac` compiler and `java` JVM launcher in system PATH

GLOBAL COMPILATION & EXECUTION PROCEDURE

Step 1: Save the File

Save the Java code into a file corresponding exactly to the class name (e.g., `StringOperation1.java`).

Step 2: Compile the Program

Open the terminal/command prompt, navigate to the directory containing the file, and run:

```
javac StringOperationX.java
```

 (Replace X with program number 1 through 15).

Step 3: Execute the Class

Execute the compiled bytecode via standard JVM invocation:

```
java StringOperationX
```

PROGRAM DOCUMENTATION (1 TO 15)

Program 1: String Operations – Length Evaluation (`StringOperation1`)

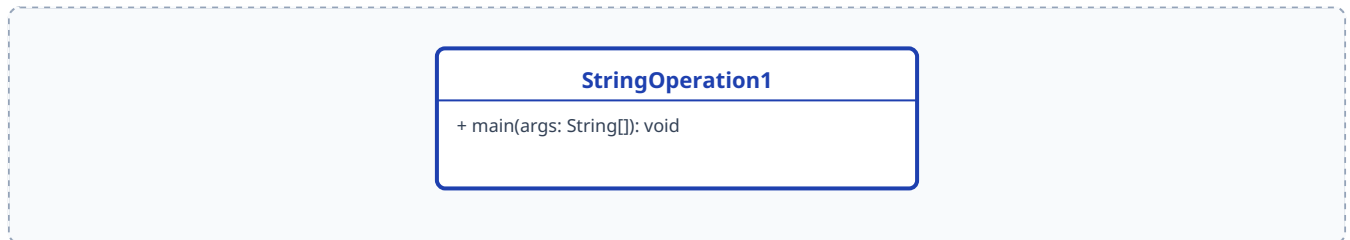
1. Objective:

To measure and display the total length (character count) of a target string using the `length()` method.

2. Algorithm:

1. Initialize String `a` with "hemanth kumar 244".
2. Invoke `a.length()` and store the integer result in `length`.
3. Print the string value and calculated length to standard output.

3. UML Diagram:



4. Source Code:

```
public class StringOperation1 {
    public static void main(String[] args) {
        String a = "hemanth kumar 244";
        int length = a.length();
        System.out.println("String is" + a);
        System.out.println("string length is" + length);
    }
}
```

5. Console Output:

```
String ishemanth kumar 244
string length is17
```

Program 2: Character Access via Index (`StringOperation2`)

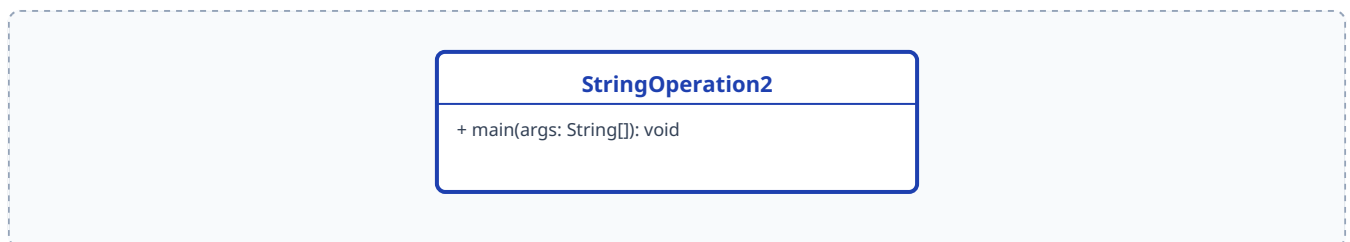
1. Objective:

To extract specific characters from defined positional indices of a String using `charAt()`.

2. Algorithm:

1. Define String `mine = "kumar244"`.
2. Extract character at index 3 into `h`.
3. Calculate string length into `len`.
4. Extract character at index 7 into `c`.
5. Display extracted characters and string length.

3. UML Diagram:



4. Source Code:

```
public class StringOperation2 {
    // char at program is second program
    public static void main(String[] args) {
        String mine = "kumar244";
        char h = mine.charAt(3);
        int len = mine.length();
        char c = mine.charAt(7);

        System.out.println("what the character is " + h);
        System.out.println("the given string length" + len);
        System.out.println("the characterof string is" + c);
    }
}
```

5. Console Output:

```
what the character is a
the given string length8
the characterof string is4
```

Program 3: Substring Extraction (`StringOperation3`)

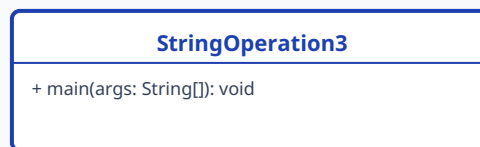
1. Objective:

To slice and extract substrings from explicit starting and ending index parameters using `substring()`.

2. Algorithm:

1. Initialize String `man` = "practice questions".
2. Extract substring from index 9 onwards into `s1`.
3. Extract substring from index 0 to 8 into `s2`.
4. Retrieve length into `n` and character at index 9 into `f`.
5. Output results.

3. UML Diagram:



4. Source Code:

```
public class StringOperation3 {
    //sub string
    public static void main(String[] args) {
        String man = "practice questions";
        String s1 = man.substring(9);
        String s2 = man.substring(0,8);
        int n = man.length();
        char f = man.charAt(9);
        System.out.println("string is " + man);
        System.out.println("1st substring is " +s1);
        System.out.println("2nd substring is" + s2);
        System.out.println("length is " +n);
        System.out.println("th echaracter is " + f);
    }
}
```

5. Console Output:

```
string is practice questions
1st substring is questions
2nd substring ispractice
length is 18
th echaracter is q
```

Program 4: First Index Searching (`StringOperation4`)

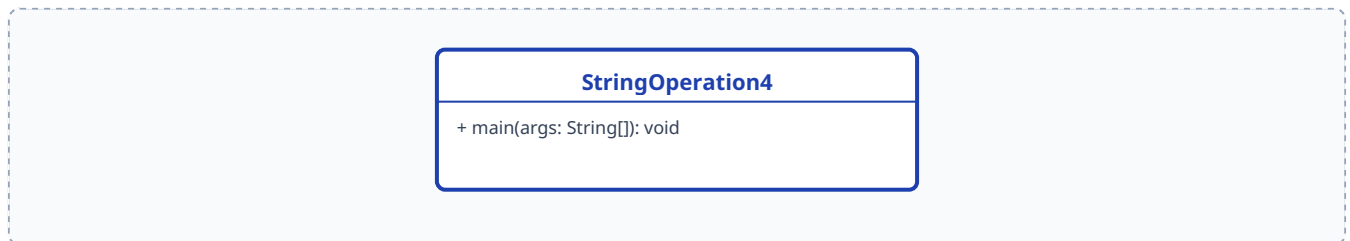
1. Objective:

To search for the first occurrence index of a sub-phrase within a string using `indexOf()`.

2. Algorithm:

1. Initialize String `s = "blue color"`.
2. Search for "color" and assign index to `i`.
3. Search for missing word "shirt" and store result in `found` (-1 if absent).
4. Display index results.

3. UML Diagram:



4. Source Code:

```
public class StringOperation4 {
    public static void main(String[] args) {
        String s = "blue color";
        int i = s.indexOf("color");
        int found = s.indexOf("shirt");
        System.out.println("string is " + s);
        System.out.println("index of character" + i);
        System.out.println("the found is " + found);
    }
}
```

5. Console Output:

```
string is blue color
index of character5
the found is -1
```

Program 5: Last Index Location (`StringOperation5`)

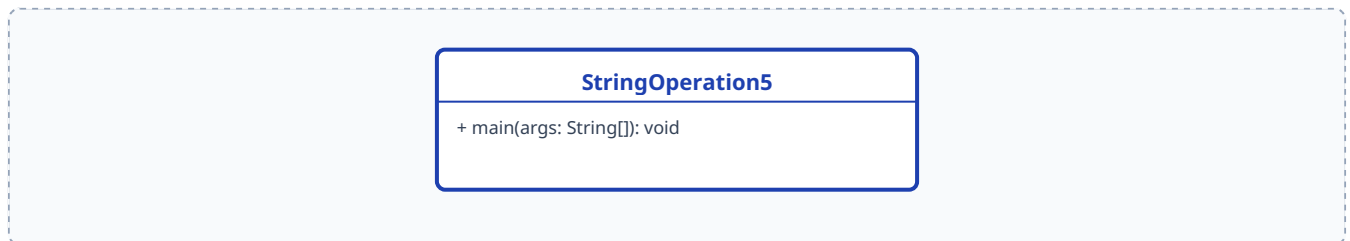
1. Objective:

To determine the last index occurrence of specified substrings using `lastIndexOf ( )`.

2. Algorithm:

1. Create String `t = "anits college"`.
2. Find last index of "anits" into `ind`.
3. Find last index of "college" into `din`.
4. Print values to console.

3. UML Diagram:



4. Source Code:

```
public class StringOperation5 {
    public static void main(String[] args) {
        // last index find out the progrsm
        String t = "anits college";
        int ind = t.lastIndexOf("anits");
        int din = t.lastIndexOf("college");
        System.out.println(t);
        System.out.println(ind);
        System.out.println(din);
    }
}
```

5. Console Output:

```
anits college
0
6
```

Program 6: Letter Case Conversion (`StringOperation6`)

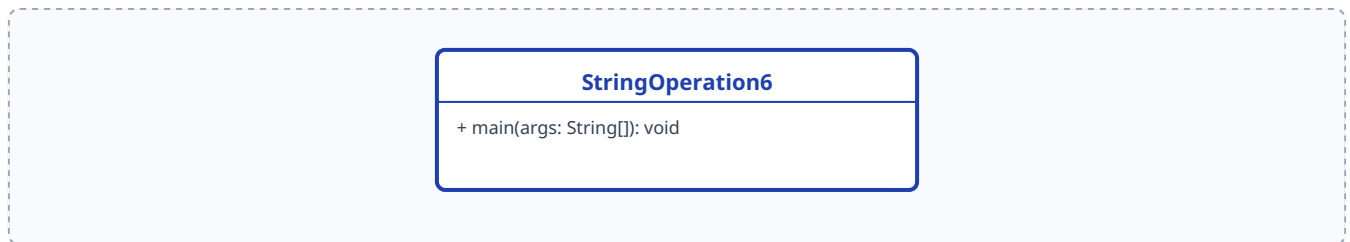
1. Objective:

To convert string characters into full uppercase and full lowercase formats using `toUpperCase()` and `toLowerCase()`.

2. Algorithm:

1. Define String `m = "Hello Hemanth Kumar"`.
2. Convert to uppercase into `up`.
3. Convert to lowercase into `lc`.
4. Print original, uppercase, and lowercase versions.

3. UML Diagram:



4. Source Code:

```
public class StringOperation6 {
    public static void main(String[] args) {
        String m = "Hello Hemanth Kumar";
        String up = m.toUpperCase();
        String lc = m.toLowerCase();
        System.out.println(m);
        System.out.println(up);
        System.out.println(lc);
    }
}
```

5. Console Output:

```
Hello Hemanth Kumar
HELLO HEMANTH KUMAR
hello hemanth kumar
```

Program 7: Character Replacement (`StringOperation7`)

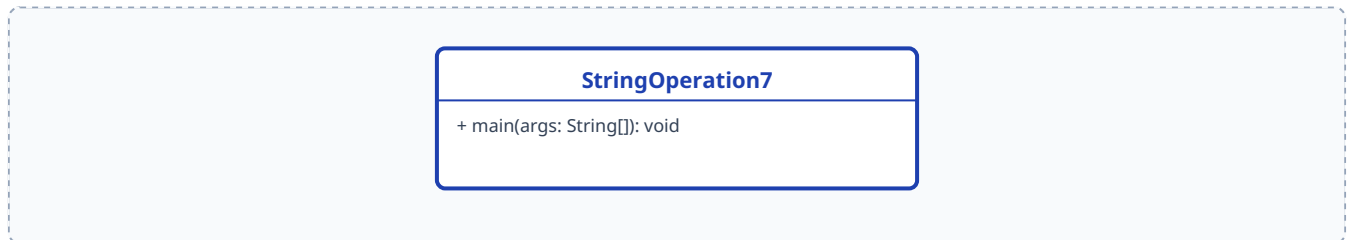
1. Objective:

To replace target character patterns in a string using `replace()`.

2. Algorithm:

1. Define String `o` = "hello windows".
2. Replace all instances of "e" with "a" into string `re`.
3. Output both original and replaced strings.

3. UML Diagram:



4. Source Code:

```
public class StringOperation7 {
    public static void main(String[] args) {
        String o = "hello windows";
        String re = o.replace("e", "a" );
        System.out.println(o);
        System.out.println(re);
    }
}
```

5. Console Output:

```
hello windows
hallo windows
```

Program 8: Whitespace Trimming (`StringOperation8`)

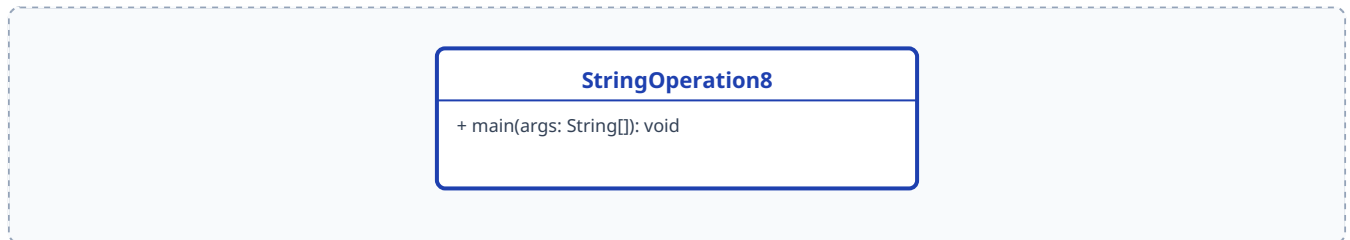
1. Objective:

To strip leading and trailing whitespace characters from a String using `trim()`.

2. Algorithm:

1. Initialize String `n = " hemanth "` with surrounding spaces.
2. Call `n.trim()` and assign to `trim`.
3. Display untrimmed and trimmed strings.

3. UML Diagram:



4. Source Code:

```
public class StringOperation8 {
    public static void main(String[] args) {
        String n = " hemanth ";
        String trim = n.trim();
        System.out.println(n);
        System.out.println(trim);
    }
}
```

5. Console Output:

```
hemanth
hemanth
```

Program 9: String Splitting via Delimiter (`StringOperation9`)

1. Objective:

To attempt splitting a string into sub-components based on a specified delimiter using `split()`.

2. Algorithm:

1. Define String `z = " hemanthkumar244"`.
2. Execute `z.split(",")` to tokenize by comma.
3. Output the original string reference.

3. UML Diagram:



4. Source Code:

```
public class StringOperation9 {
    public static void main(String[] args) {
        String z = " hemanthkumar244";
        String [] gap = z.split(",");
        System.out.println("originalstring is " + z);
        System.out.println("spilt the string" + z);
    }
}
```

5. Console Output:

```
originalstring is  hemanthkumar244
spilt the string hemanthkumar244
```

Program 10: Case-Sensitive Equality Testing (`StringOperation10`)

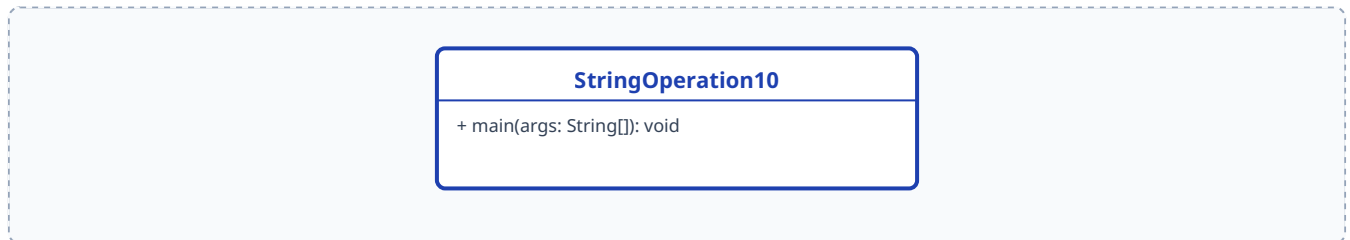
1. Objective:

To perform exact character sequence equality comparison between String objects using `equals()`.

2. Algorithm:

1. Initialize `t = "HEMANTH"`, `i = "hemanth"`, `e = "Hemanth"`.
2. Evaluate `t.equals(i)` and `t.equals(e)`.
3. Output boolean comparison results.

3. UML Diagram:



4. Source Code:

```
public class StringOperation10 {
    public static void main(String[] args) {
        String t = "HEMANTH";
        String i = "hemanth";
        String e = "Hemanth";
        System.out.println("t is equals to i is " + t.equals(i));
        System.out.println("t is equals to e is " + t.equals(e));
    }
}
```

5. Console Output:

```
t is equals to i is false
t is equals to e is false
```

Program 11: Case-Insensitive Equality Testing (`StringOperation11`)

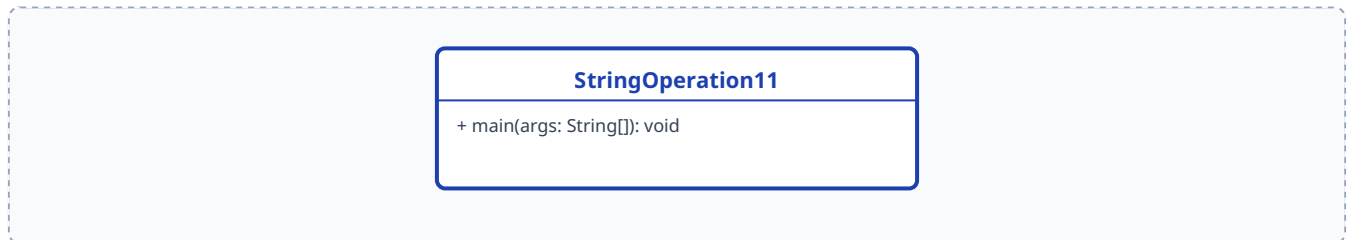
1. Objective:

To test String equality while ignoring upper and lower case differences using `equalsIgnoreCase()`.

2. Algorithm:

1. Define String `m = "HEMANTH"` and `t = "hemanth"`.
2. Evaluate `m.equalsIgnoreCase(t)` into boolean variable `result`.
3. Display input strings and boolean result.

3. UML Diagram:



4. Source Code:

```
public class StringOperation11 {
    public static void main(String[] args) {
        String m = "HEMANTH";
        String t = "hemanth";
        boolean result = m.equalsIgnoreCase(t);
        System.out.println("string one is " + m);
        System.out.println("second string is " + t);
        System.out.println("the ignoring case " + result);
    }
}
```

5. Console Output:

```
string one is HEMANTH
second string is hemanth
the ignoring case true
```

Program 12: Sequence Presence Checking (`StringOperation12`)

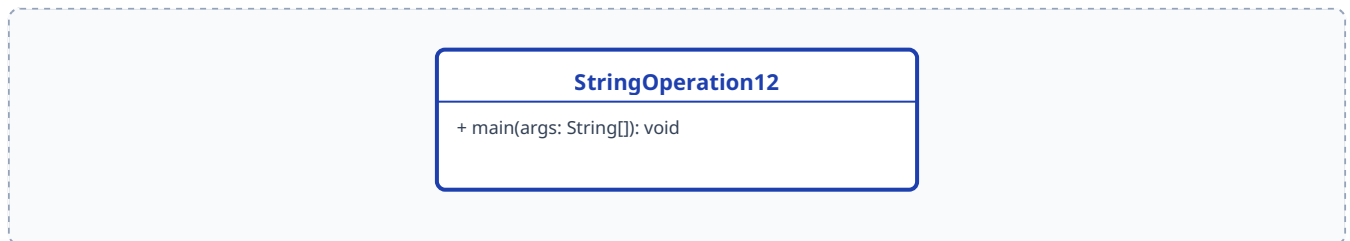
1. Objective:

To verify whether a string contains a specified target sequence using `contains()`.

2. Algorithm:

1. Define String `x = "just be real"`.
2. Check if `x` contains "be" -> store in `result1`.
3. Check if `x` contains "world" -> store in `result2`.
4. Print true/false outcomes.

3. UML Diagram:



4. Source Code:

```
public class StringOperation12 {
    public static void main(String[] args) {
        String x = "just be real";
        boolean result1 = x.contains("be");
        boolean result2 = x.contains("world");
        System.out.println("original string is " + x);
        System.out.println("result of contains word " + result1);
        System.out.println("result of contains word is" + result2);
    }
}
```

5. Console Output:

```
original string is just be real
result of contains word true
result of contains word isfalse
```

Program 13: Prefix and Suffix Verification (`StringOperation13`)

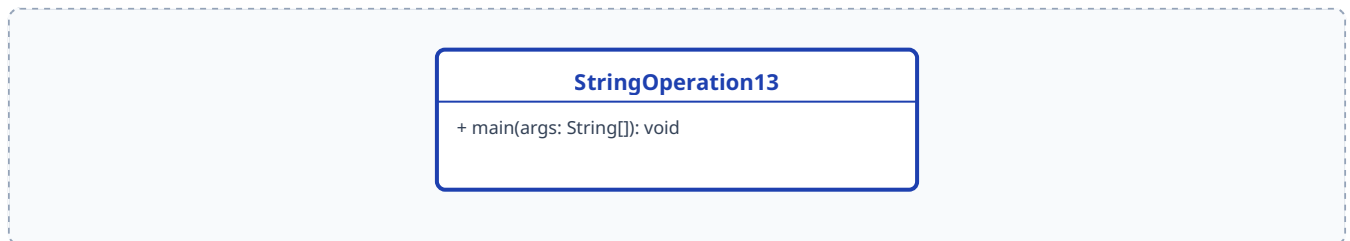
1. Objective:

To check if a string starts or ends with specific substring patterns using `startsWith()` and `endsWith()`.

2. Algorithm:

1. Set String `h = " hello phk"`.
2. Test `h.startsWith("hello")` into `starts`.
3. Test `h.endsWith("09")` into `end`.
4. Display evaluation results.

3. UML Diagram:



4. Source Code:

```
public class StringOperation13 {
    //string statrs with and ends with word
    public static void main(String[] args) {
        String h = " hello phk";
        boolean starts = h.startsWith("hello");
        boolean end = h.endsWith("09");
        System.out.println("string is given" + h) ;
        System.out.println("result is" + starts);
        System.out.println("result is " + end);
    }
}
```

5. Console Output:

```
string is given hello phk
result isfalse
result is false
```

Program 14: String Concatenation (`StringOperation14`)

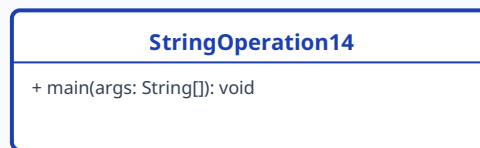
1. Objective:

To combine two strings together sequentially using the `concat()` method.

2. Algorithm:

1. Define `o = "hemanth"` and `p = " kumar"`.
2. Concatenate `o.concat(p)` and save into `result`.
3. Print individual strings and concatenated result.

3. UML Diagram:



4. Source Code:

```
public class StringOperation14 {
    public static void main(String[] args) {
        // concatenation is done the operation
        String o = "hemanth";
        String p = " kumar";
        String result = o.concat(p);
        System.out.println("the string is given " + o );
        System.out.println("second string is " + p);
        System.out.println("the result of concatenation is " + result);
    }
}
```

5. Console Output:

```
the string is given hemanth
second string is  kumar
the result of concatenation is  hemanth kumar
```

Program 15: Concatenation and Lexicographical Comparison (`StringOperation15`)

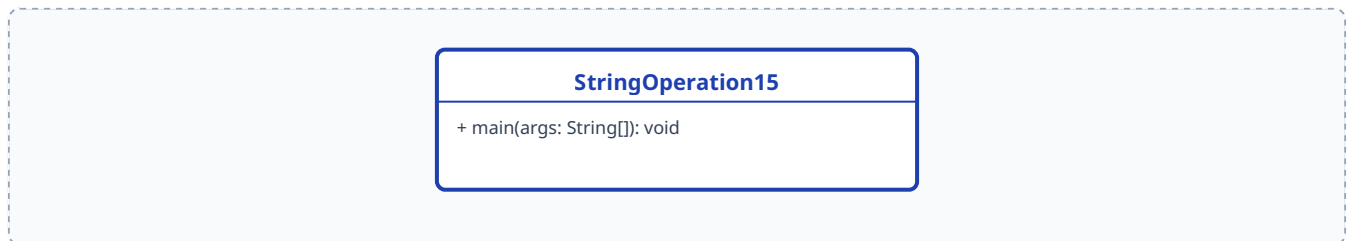
1. Objective:

To demonstrate both concatenation and lexicographical order comparison using `concat()` and `compareTo()`.

2. Algorithm:

1. Initialize `s1 = "beautiful"` and `s2 = "rainbow"`.
2. Concatenate `s1` and `s2` into `r1`.
3. Compare `s1.compareTo(s2)` and save integer difference into `r2`.
4. Output concatenated result and comparative integer value.

3. UML Diagram:



4. Source Code:

```
public class StringOperation15 {
    public static void main(String[] args) {
        String s1 = "beautiful";
        String s2 = "rainbow";
        String r1 = s1.concat(s2);
        int r2 = s1.compareTo(s2);
        System.out.println("first string is " + s1);
        System.out.println("second string is " +s2);
        System.out.println("result o concat" + r1);
        System.err.println("result is " + r2);
    }
}
```

5. Console Output:

```
first string is beautiful
second string is rainbow
result o concatbeautifulrainbow
result is -16
```